

L o D

Law of Demeter

Principle

Only talk to your immediate friends

DEFINITION

An object should only call methods on itself, its fields, its parameters, and objects it creates — not reach through chains.

The Law of Demeter limits who a method may talk to: itself, its own fields, its parameters, and objects it creates — but not objects returned by those calls. "Tell, don't ask" is the practical form: instead of pulling data out of a graph to act on it, send a message to the object that owns the data and let it act. It is a heuristic about coupling, not a literal ban on dots.

WHY IT MATTERS

A chain like `a.b().c().d()` hard-wires the caller to the shape of an entire object graph it does not own; a change two hops away breaks code that never mentioned that class, and every hop is another chance to hit a null. Keeping to direct collaborators preserves encapsulation — objects stay free to change their internals — and localizes the damage when they do.

— How it works

Direct collaborator An object a method may legitimately talk to: this, its fields, its parameters, and objects it creates. One hop is allowed.

Stranger An object reached only through a collaborator's return value. Calling it — "reaching through" — is the violation LoD warns about.

Intention method A method on the owner that does the work for you (`order.shipToCity()`), hiding the navigation behind one trusted call.

HOW TO APPLY

1. Spot the train wrecks: expressions that chain across different objects' return values.
2. Ask what the caller is trying to do, then name a method for that intent on the object it holds.
3. Move the navigation inside that owner ("tell, don't ask"), exposing intent instead of structure.
4. Stop at one hop; if you need a delegate method, add it deliberately, not as a reflex.

LITMUS TEST

Count the dots that walk across objects. Each extra hop is another class you are coupled to and another place to get a null halfway down.

— Smell → fix

Reaching through three objects just to read a city:

```
const city = order.getCustomer().getAddress().getCity()
ship(city)
// coupled to Customer AND Address internals; either change breaks this
```

↓ TELL, DON'T ASK

```
order.shipToCity() // ask the object you hold to do the work
// inside Order: shipToCity() { return this.customer.shipToCity() }
```

Push the behavior to the object that owns the data. Order exposes intent (shipToCity) and hides the navigation behind one trusted hop.

USE WHEN

- ✓ Reducing coupling to a deep object graph
- ✓ Encapsulating navigation behind intent

AVOID WHEN

- ▲ Adding endless delegate methods that bloat the API

— Trade-offs & pitfalls

PROS

- + Lower coupling
- + Refactor-friendly

CONS

- Delegation boilerplate
- Fluent APIs legitimately chain

COMMON PITFALLS

- ▲ Dot-counting zealotry: wrapping everything in delegate methods until the API bloats — LoD is about coupling, not punctuation.
- ▲ Confusing fluent or builder chains (same object/type) with train wrecks (foreign objects).
- ▲ Adding pass-through getters that re-expose the very internals you were hiding.

WHERE YOU SEE IT

- ▶ `order.shipToCity()` instead of `order.getCustomer().getAddress().getCity()`.
- ▶ A service method that takes exactly the value it needs, not a god object to dig through.
- ▶ An Angular component talking to its own injected service, not reaching into that service's collaborators.

SEE ALSO

Information Expert	GRASP: assign the work to the object holding the information
Col	delegating to a held collaborator is the composition view of LoD
SoC	keeping navigation inside the owner separates concerns
SRP	the object that has the data also owns the operation on it

— Interview & sources

Does LoD ban all method chaining?

No. Fluent and builder APIs return the same object or type and are fine. LoD targets reaching through foreign object graphs you do not own.

What is the cost of following it too literally?

Delegation boilerplate — wrapper methods that just forward calls. Apply it to cut real coupling, not as a dot-counting rule.

How does LoD relate to encapsulation?

It enforces it: talking only to direct collaborators stops you depending on others' internal structure, so they stay free to change.

Does an ORM or query builder violate LoD?

Usually no — fluent builders return the same type and ORMs expose intent methods. The violation is reaching through unrelated foreign objects, not chaining on one cohesive API.

REMEMBER

Talk only to your immediate neighbors; don't reach through them to strangers.

SOURCES

Karl Lieberherr et al. — the Law of Demeter, Northeastern University (1987)

Andrew Hunt, David Thomas — "The Pragmatic Programmer" (1999): decoupling and the Law of Demeter